

Data Analysis for Experimental Physics with Machine Learning

A.A. 2025/26, Alessandro Foglizzo

1 Concetti fondamentali

- Dataset: insieme di coppie (x_i, y_i) , per $i=1\dots N$ eventi
- Features: insieme di variabili (indice $j=1\dots m$ features) che caratterizzano un evento i : $x_i = (x_{i1}, \dots, x_{ij}, \dots, x_{im})$
- Label: etichetta associata all'evento, è la variabile che lo classifica e viene utilizzata per il training, è il risultato atteso
- Modello: funzione matematica che, prendendo in pasto le features, attraverso i parametri θ ottenuti durante il training, ottiene la predizione su $\hat{y}$
- Funzione oggettiva: $\text{obj}(\theta) = L(\theta) + \Omega(\theta)$, misura il grado di accordo tra il fit e i dati. es MSE
- Loss function $L(\theta)$: misura quanto il modello è predittivo rispetto al training set $L(\theta) = \sum_i (y_i - \hat{y}_i)^2$
- Regularization term $\Omega(\theta)$: controlla la complessità del modello e aiuta ad evitare l'overfitting
- variance-bias tradeoff: equilibrio tra un modello predittivo e semplice
- Training error: $E_{out} = Bias^2 + Var + Noise$
- Varianza: misura quanto cambia la predizione del modello cambiando il dataset di apprendimento, e quindi la sensibilità del modello alle fluttuazioni statistiche del dataset
- Bias: o distorsione, indica la differenza tra le assunzioni sulla semplicità del modello e la complessità dei dati reali, ed è la differenza tra il valore predetto e quello reale
- Overfitting: alta varianza e basso bias, dato da modelli troppo complessi. Errore di training molto basso, errore di test molto alto (non generalizza)
- Underfitting: alto bias e bassa varianza, dato da modelli troppo semplici. Errore di training e di test molto alti.

2 Neural Networks

Estrazione automatica delle features per trovare quelle migliori per un dato task.

Ogni neurone, sommando i pesi associati a ciascuna feature, misura una distanza da un iperpiano che viene non-linearizzata per esaltarne le differenze (4.5).

Diversi tipi di non linearità: Perceptron (θ), sigmoide, tanh, ReLU, Leaky ReLU, ELU (4.6).

L'input dei neuroni del layer denso n ($z = W^T x$) è dato dall'output (attivazioni, $a = \sigma(z)$) dei neuroni del layer $n - 1$, ed ha pesi indipendenti. Tutti i layer, meno l'input e l'output, sono nascosti (hidden): **Feed Forward Fully Connected Neural Network**. L'output dipende dal task:

- Classificazione binaria: output singolo, con attivazione a sigmoide
- Regressione lineare 1D: singolo neurone con attivazione lineare
- Multiclassification: K neuroni con attivazione softmax

Senza le non linearità, i vari strati sarebbero semplificabili a una mera moltiplicazione di matrici, che dà una matrice (lineare).

Universal Approximation Theorem: data una funzione $y = f(x)$ e un $\varepsilon > 0$, esiste un deep network $y = f_w(x)$ di profondità o larghezza arbitraria per cui

$$\sup ||f(x) - f_w(x)|| < \varepsilon$$

Quindi, una NN sufficientemente larga o profonda può approssimare una qualsiasi funzione continua su un dominio compatto. Questo però non garantisce che sia *learnable*.

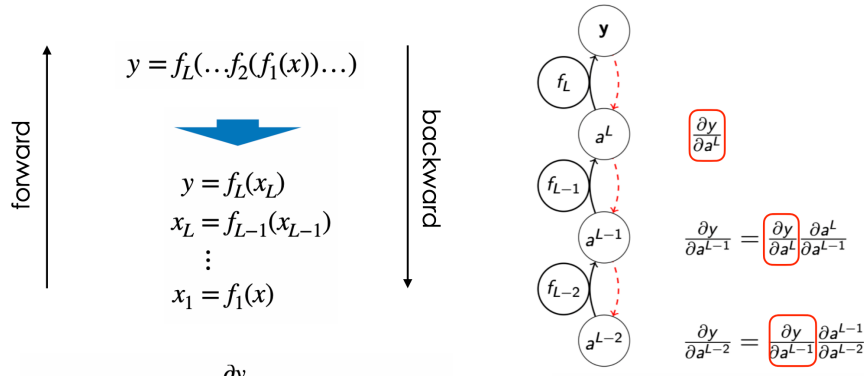
Training NN: a definita la LOSS FUNCTION in base al task e alle assunzioni: **cross-entropy** per classificazione, **MSE** per regressione. Dopodiché, trovare i pesi $\hat{w}$ che massimizzano la massima verosimiglianza $L(w)$. Per il *gradient descent*: $\frac{\partial L(w)}{\partial w}$.

Backward Propagation Algorithm: è l'algoritmo per calcolare i gradienti delle loss function. Sappiamo che che

$$y = f_L(\dots f_2(f_1(x))\dots)$$

con

$$y = f_L(x_L) \text{ e } x_L = f_{L-1}(x_{L-1})$$

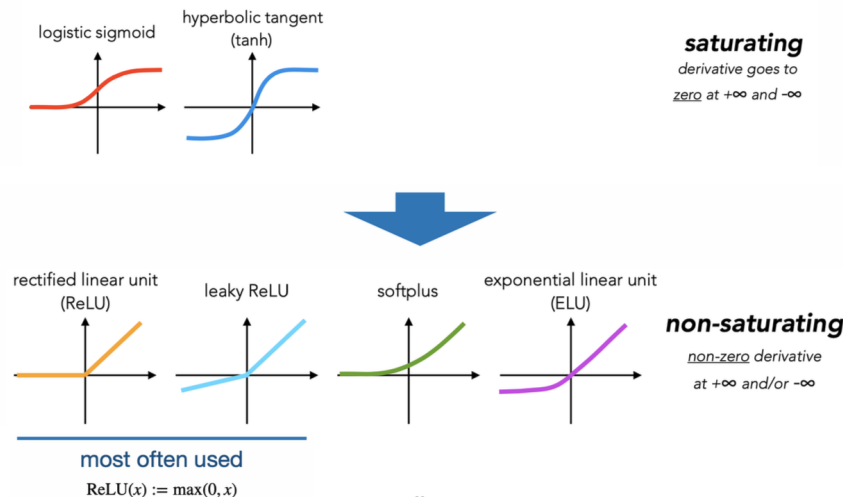


Per calcolare $\frac{\partial y}{\partial x_l}$ con $l \in 1, \dots, L$ dobbiamo applicare la chain rule. Partendo quindi dalla derivata all'output $y = f_L$, calcoliamo a ritroso tutte le $\frac{\partial y}{\partial x_l}$. (4.14)

La backpropagation è fatta in automatico dalla maggior parte dei pacchetti di deep learning. L'unica condizione per poter definire un tipo di layer è saperne definire il gradiente.

Le non linearità *saturating* hanno la derivata molto piccola praticamente ovunque, quindi mandano il gradiente a zero. Inoltre la sigmoide è problematica perché non è centrata in zero e contiene un e^x , lento computazionalmente.

Al contrario, la ReLU è molto usata e fornisce rapida convergenza e computazione; non è però centrata in zero (possibile instabilità, vedere batch normalization).



L'obiettivo dell'**inizializzazioni dei pesi** è mantenere attivazioni stabili lungo i vari layer. Vorremmo che $\text{Var}(a) \approx \text{Var}(z)$ per evitare che i segnali esplodano o svaniscano (vanishing gradients) durante la propagazione. Notazione:

- W : matrice dei pesi
- $\mathcal{N}(\mu, \sigma^2)$: distribuzione normale con media μ e varianza σ^2
- $\sim$: estratto dalla distribuzione
- $n_{in/out}$: numero di neuroni in ingresso/uscita del layer (fan in/out)

Esistono diverse inizializzazioni:

- **Zero Initialization:** mette tutti i pesi a zero, da evitare per problemi di simmetria che impediscono l'apprendimento
- **Lecun Initialization:** $W \sim \mathcal{N}(0, \frac{1}{n_{in}})$
- **He (Kaiming) Initialization:** progettata per ReLU e Leaky ReLU, $W \sim \mathcal{N}(0, \frac{2}{n_{in}})$ (il fattore 2 compensa l'azzeramento, in media, della metà delle attivazioni)
- **Xavier (Glorot) Initialization:** progettata per sigmoide e tanh, $W \sim \mathcal{N}(0, \frac{1}{n_{in}+n_{out}})$ (cerca un bilanciamento tra il passo avanti e quello indietro)